



UNIVERSITÀ DI PADOVA

Dipartimento
di Ingegneria
dell'Informazione

Motion Estimation

Marco Cagnazzo

Multimedia Communications

Outline

Introduction

Variational methods

Block-matching methods

Criteria & Strategy

Advanced Techniques

Parametric methods

Deep Learning methods

Conclusions



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Outline



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Introduction

Variational methods

Block-matching methods

Criteria & Strategy

Advanced Techniques

Parametric methods

Deep Learning methods

Conclusions



Introduction: movement

- ▶ A video sequence is very rich in information content
- ▶ Motion conveys the majority of this information
 - ▶ Movement allows objects identification
 - ▶ Image characteristics are coherent along motion trajectories
- ▶ *Motion detection*: binary decision (motion or no-motion)
- ▶ *Motion estimation*: measure the movement

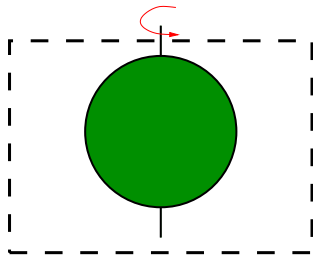


Motion and apparent motion

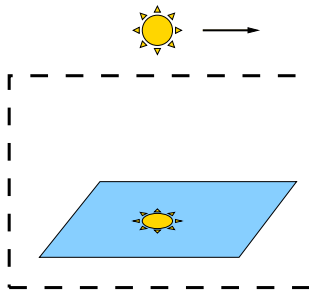
- ▶ 2D motion field:
 - ▶ Projection of physical movement into the image plane
- ▶ Optical flow:
 - ▶ Apparent motion of a luminance pattern
- ▶ The motion field and the optical flow often are the same but this is not necessarily always true



Motion and apparent motion



Purely rotating ball: Null optical flow, real motion is not null



Reflecting surface illuminated by moving source: The optical flow is not null, but there is no physical motion



Motion estimation applications

- ▶ Processing
 - ▶ Segmentation
 - ▶ Denoising
 - ▶ Restoration
 - ▶ Frame-rate up-conversion
 - ▶ Video surveillance / video protection
- ▶ Compression
 - ▶ Current image prediction from encoded images
 - ▶ Trade-off between complexity, precision and coding cost
 - ▶ What is the relationship between the estimated and the “real” motion?
- ▶ Complexity

Outline

Introduction

Variational methods

Block-matching methods

Criteria & Strategy

Advanced Techniques

Parametric methods

Deep Learning methods

Conclusions



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Variational methods

- ▶ Variational methods are designed to estimate the underlying physical motion
- ▶ They result into a velocity vector field: the **optical flow**
- ▶ The mathematical structure is a 2D vector field, referred to as $\mathbf{V}(x, y)$:

$$\mathbf{V} : (x, y) \in \mathcal{J} \subset \mathbb{R}^2 \rightarrow (u, v)$$

- ▶ $\mathcal{I} \subset \mathbb{R}^2$ is the image support (coordinates in the image plane)
- ▶ (x, y) is a point in the image plane
- ▶ $(u(x, y), v(x, y))$ is the velocity of this point
- ▶ This formulation uses real-values variables (x, y) : thus, in practical applications, we need to define a discretization process

Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione





UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Video Sequence Example



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione





UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Video Sequence Example



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione





UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Video Sequence Example





UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Video Sequence Example



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione





UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Video Sequence Example



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione





UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Video Sequence Example





UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Video Sequence Example





UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Video Sequence Example



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Video Sequence Example



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione





Motion Vector Field (MVF) Example



Observations:

- ▶ Dense or Sparse representation.
- ▶ Arrows indicate direction and magnitude.
- ▶ Used as ground truth or estimation result.



Variational methods

- ▶ The problem of determining the optical flow is formulated as an optimization problem with *data attachment* and *regularization* terms
- ▶ Typical formulation:

$$\arg \min_{u,v} \{ D[f_0, f_1, (u, v)] + R(u, v) \}$$

- ▶ f_0 and f_1 are the two input images, (u, v) is the estimated motion field with $u(x, y)$ and $v(x, y)$ being the horizontal and vertical velocities
- ▶ D is the data attachment term: it measures how much the estimated field is coherent with the data f_0 and f_1
- ▶ R is the regularization term. It does not depend on data and it is used to penalize solutions that deviated from our *a priori* knowledge of the problem



Optical flow: problem statement

- ▶ Motion is estimated on the basis of brightness variations (Y component)
- ▶ Let us consider a generic point $\mathbf{p}$ in the image coordinates (2D): $\mathbf{p} = [x \ y]^T$
- ▶ A point on the plane corresponds to a point of an object
- ▶ For each point we consider the *trajectory* $x(t)$ passing through it at a given time as the temporal evolution of the positions of the same point on the physical object
- ▶ We want to estimate the velocity of a generic trajectory at time t and position $\mathbf{p}$: this is the **optical flow**
- ▶ The associate displacement in a given interval (e.g., between two consecutive frames) is referred to as **motion vector field**



Optical flow: the displacement field

- ▶ Let us consider a (point of an) object that, during an interval of duration T (delay), moves from $\mathbf{p} - \mathbf{D}$ to $\mathbf{p}$
- ▶ Its trajectory is $\mathbf{x}(t)$ and

$$\mathbf{x}(t_0) = \mathbf{p} - \mathbf{D}$$

$$\mathbf{x}(t_0 + T) = \mathbf{p}$$

$$\mathbf{D}(\mathbf{p}, t_0, T) = \mathbf{x}(t_0 + T) - \mathbf{x}(t_0) = \begin{bmatrix} c(\mathbf{p}, t_0, T) \\ d(\mathbf{p}, t_0, T) \end{bmatrix} = \begin{bmatrix} c(x, y) \\ d(x, y) \end{bmatrix}$$

- ▶ $\mathbf{D}(\mathbf{p}, t, T)$ is the displacement field and $c(\mathbf{p}, t, T)$, $d(\mathbf{p}, t, T)$ are its components
- ▶ For the sake of simplicity, the dependency of $\mathbf{D}$, c , d on time and delay is not shown explicitly in the following





Optical flow: the OF equation

The **velocity** field (or, optical flow) is defined as:

$$\mathbf{V}(x, y) = \lim_{T \rightarrow 0} \frac{\mathbf{D}(x, y)}{T} = \begin{bmatrix} u(x, y) \\ v(x, y) \end{bmatrix}$$

From (1), we apply Taylor expansion, divide by T and compute the limit for $T \rightarrow 0$:

$$\begin{aligned} f(p, t + T) &= f(p, t) - [c(p)f_x(p, t) + d(p)f_y(p, t)] + o[\|\mathbf{D}(p)\|] \\ \frac{f(p, t + T) - f(p, t)}{T} &= -\frac{\mathbf{D} \nabla f}{T} + \frac{o[\|\mathbf{D}(p)\|]}{T} \\ f_t &= -\mathbf{V} \nabla f \end{aligned}$$

$$uf_x + vf_y + f_t = 0$$



Optical flow: the OF equation

► Equation

$$uf_x + vf_y + f_t = 0$$

is called *Optical flow* equation

- It can be discretized for practical applications
- Several motion estimation techniques are based on it
- Nevertheless, we have only one equation and two unknowns, u and v
- Thus, it only allows to compute one component of the velocity field (the one parallel to the image spatial gradient)



Optical flow: solving the OF equation

- ▶ Additional constraints are needed to solve the OF equation
- ▶ Moreover, noise and other degradations make this equation impossible to be perfectly satisfied
- ▶ Solution: we **minimize** the energy of $uf_x + vf_y + f_t$ (instead of setting it to zero) under suitable constraints







OF: the Horn and Schunck algorithm

Setting:

$$\frac{\partial \Theta}{\partial \omega} - \frac{\partial^2 \Theta}{\partial x \partial \omega_x} - \frac{\partial^2 \Theta}{\partial y \partial \omega_y} = 0$$

with $\omega = u$ and $\omega = v$, we get respectively:

$$\lambda \nabla_2 u = (uf_x + vf_y + f_t)f_x$$

$$\lambda \nabla_2 v = (uf_x + vf_y + f_t)f_y$$

Two equations in two unknowns!



OF: the Horn and Schunck algorithm

The Laplacian ∇u can be approximated as $\bar{u} - u$, where $\bar{u}$ is a local average of u values. One gets

$$\begin{cases} \lambda(\bar{u} - u) = (uf_x + vf_y + f_t)f_x \\ \lambda(\bar{v} - v) = (uf_x + vf_y + f_t)f_y \end{cases} \quad \begin{cases} (f_x^2 + \lambda)u + f_y f_t v = \lambda \bar{u} - f_t f_x \\ f_x f_t u + (f_y^2 + \lambda)v = \lambda \bar{v} - f_t f_y \end{cases} \quad \begin{cases} u = \bar{u} - f_x \frac{\bar{u} f_x + \bar{v} f_y + f_t}{\lambda + \|\nabla f\|^2} \\ v = \bar{v} - f_y \frac{\bar{u} f_x + \bar{v} f_y + f_t}{\lambda + \|\nabla f\|^2} \end{cases}$$

However, $\bar{u}$ depends on u and $\bar{v}$ depends on v . In order to find a solution, Horn and Schunck proposed an iterative algorithm, which requires an initialization

$$\begin{cases} u^{n+1} = \bar{u}^n - f_x \frac{\bar{u}^n f_x + \bar{v}^n f_y + f_t}{\lambda + \|\nabla f\|^2} \\ v^{n+1} = \bar{v}^n - f_y \frac{\bar{u}^n f_x + \bar{v}^n f_y + f_t}{\lambda + \|\nabla f\|^2} \end{cases}$$



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

OF: the Horn and Schunck algorithm

Example





OF: the Horn and Schunck algorithm

Example

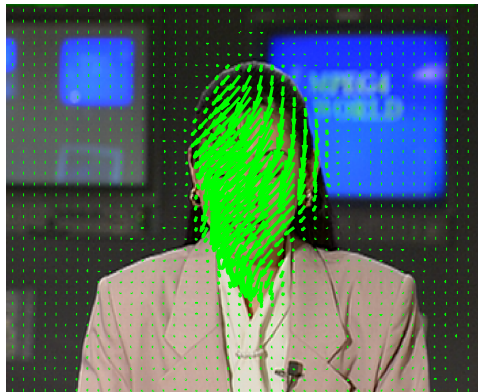


OF: the Horn and Schunck algorithm



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Example





Outline

Introduction

Variational methods

Block-matching methods

Criteria & Strategy

Advanced Techniques

Parametric methods

Deep Learning methods

Conclusions



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione





Block matching

- ▶ Block matching is extremely popular in video compression
- ▶ Trade-off between precision and coding cost of motion description
 - ▶ Number of blocks per image (block size)
 - ▶ Number of parameters per block
 - ▶ Number of bits per parameter
- ▶ Complexity can grow very large

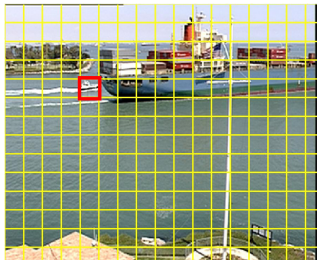


Block matching

Notation

- ▶ Let us consider a $N \times M$ image
- ▶ A block $B_{p,q}$ is defined as the *set of indexes* starting in (p, q) , whose size is $P \times Q$:

$$B_{p,q} = \{p, p+1, \dots, p+P-1\} \times \{q, q+1, \dots, q+Q-1\}$$





Block matching

Notation

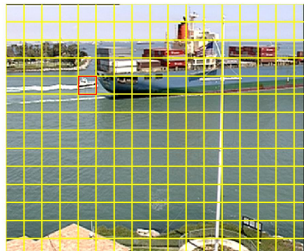
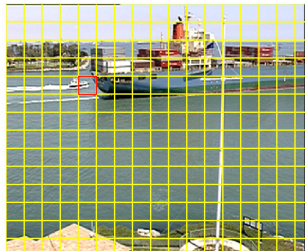
- ▶ We introduce the symbol $\mathbf{f}_k(B_{p,q})$ to indicate the vector of PQ luminance values in the positions $B_{p,q}$ at time k :

$$\mathbf{f}_k(B_{p,q}) = [f(p, q, k), f(p+1, q, k), \dots, \\ f(p+P-1, q, k), f(p, q+1, k), \dots, \\ f(p+P-1, q+Q-1, k)]^T$$

- ▶ The block matching consists in computing a *dissimilarity measure* between $\mathbf{f}_k(B_{p,q})$ and $\mathbf{f}_h(B_{p-i, q-j})$
- ▶ The vector (i, j) minimizing this dissimilarity (or distance) is selected

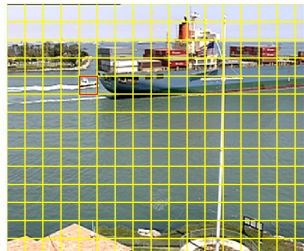
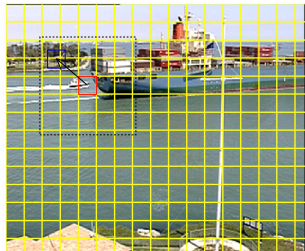


Block matching



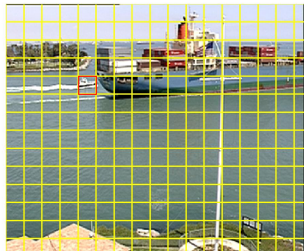
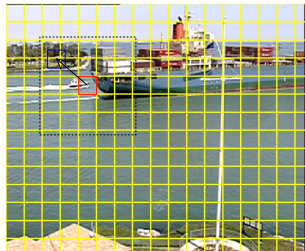


Block matching





Block matching



We compare $\mathbf{f}_k(B_{p,q})$ and $\mathbf{f}_h(B_{p-i,q-j})$



Block matching

Notation

In summary, block matching consists in solving, for each block $B_{p,q}$ of current image k , the following problem:

$$\left(\hat{i}, \hat{j}\right) = \arg \min_{(i, j) \in \mathcal{W}} d\left[\mathbf{f}_k\left(B_{p, q}\right), \mathbf{f}_h\left(B_{p-i, q-j}\right)\right] = \arg \min_{(i, j) \in \mathcal{W}} J(i, j)$$

BM techniques differ in:

- ▶ The minimization criterion d (or J)
- ▶ The set $\mathcal{W}$ of candidate vectors (search window) and the strategy for scanning $\mathcal{W}$
- ▶ The block size and shape



Block matching

- Therefore, the estimated motion vector field (MVF) (u, v) is a piece-wise constant vector obeying the following:

$$\forall (n, m) \in B_{p,q}, (u(n, m), v(n, m)) = \arg \min_{(i,j) \in \mathcal{W}} d[\mathbf{f}_k(B_{p,q}), \mathbf{f}_h(B_{p-i,q-j})]$$

- ▶ A more precise (but also heavier) notation would be $(u_{h \rightarrow k}(n, m), v_{h \rightarrow k}(n, m))$, highlighting both the *reference* frame k and the *current* frame h
- ▶ For example, when $h = k - 1$ we are estimating the *forward* motion from the previous image
- ▶ We can also estimate the *backward* motion by setting $h = k + 1$
- ▶ However, we will drop the subscript $h \rightarrow k$ when not necessary



Evaluation of the MVF

- ▶ What is a “good” MVF? We need to define some cost and performance criteria for the resulting MVF
- ▶ In order to define the *quality* of the MVF, we compute the **energy of the prediction error**
- ▶ This means that we use the MVF to compute a prediction of image k from image h
- ▶ The prediction is also referred to as **motion-compensated** image or prediction
- ▶ Then we compute the mean square value of the difference (error) between the prediction and the true image h



Evaluation of the MVF

- ▶ In formulas, the motion-compensated (MC-ed) prediction associated to the MVF ($u_{h \rightarrow k}, v_{h \rightarrow k}$) is:

$$\tilde{f}_k(n, m) = f_h(n + u_{h \rightarrow k}(n, m), m + v_{h \rightarrow k}(n, m))$$

- ▶ The prediction error is:

$$e(n, m) = f_k(n, m) - \tilde{f}_k(n, m)$$

- ▶ The MC-ed MSE is:

$$\mathcal{E} = \frac{1}{NM} \sum_{n,m} e^2(n, m)$$

- ▶ As usual, an equivalent metric is the PSNR:

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\mathcal{E}}$$



Evaluation of the MVF

- ▶ The PSNR is a quality or performance measure of the MVF
- ▶ A cost measure can be the number of bits needed to losslessly encode the MVF
- ▶ It is called the *coding cost*
- ▶ The coding cost depends on the technique used to encode the vectors:
 - ▶ Fixed-length coding
 - ▶ Exp-Golomb coding (possibly, with prediction)
 - ▶ Huffman coding
 - ▶ Arithmetic coding (possibly, context-based)
- ▶ In order to present a measure that is related to the MVF rather than to the technique, we can use the MVF empirical entropy
- ▶ It amounts to estimate the probability of MVF components via their relative frequency in (u, v) , and to use this probability in the entropy formula



Evaluation of the MVF

- ▶ Last but not least, when deciding the parameters of a ME method, we shall consider the computational complexity
- ▶ In order to compute $\arg \min_{(i,j) \in \mathcal{W}} d[f_k(B_{p,q}), f_h(B_{p-i,q-j})]$, we need:
 - ▶ For each block $B(p, q)$
 - ▶ and for each candidate vector $(i, j) \in \mathcal{W}$
 - ▶ compute one instance of $d[f_k(B_{p,q}), f_h(B_{p-i,q-j})]$
- ▶ Therefore the following choices impact on the computational complexity:
 - ▶ The block size (which determines the number of blocks)
 - ▶ The number of candidate vectors (which depends on the search window *and* on the search strategy)
 - ▶ The cost function (or criterion) d
- ▶ For each of these parameters' choice, there is a trade-off between complexity, quality and coding cost of the MVF
- ▶ Understanding this trade-off is of fundamental importance to design an effective ME technique



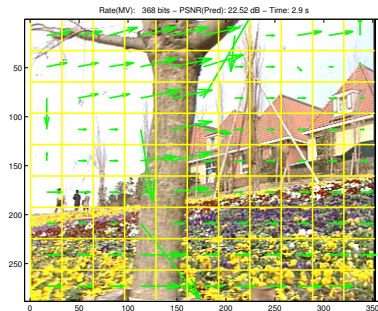
Evaluation of the MVF

- ▶ In order to understand this trade-off, let us consider the block-size $P \times Q$
- ▶ A large block size will typically:
 - ▶ reduce the complexity (less blocks)
 - ▶ reduce the coding cost (for the same reasons, less blocks means less vectors to encode)
 - ▶ increase the MSE, because we use a less flexible mathematical model: the MVF is constant on large areas
- ▶ So, what is the ideal block size?

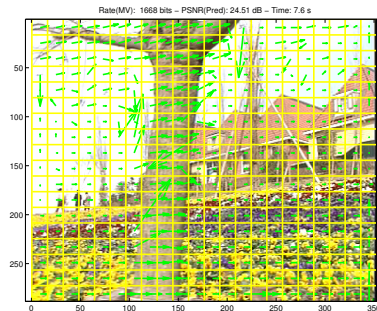


Parameters of motion estimation

Block size and shape



Block size 32×32



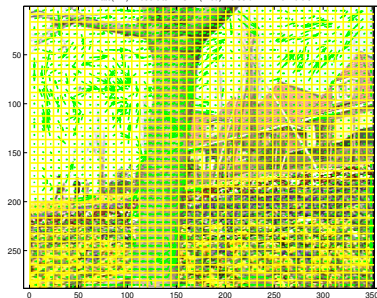
Block size 16×16



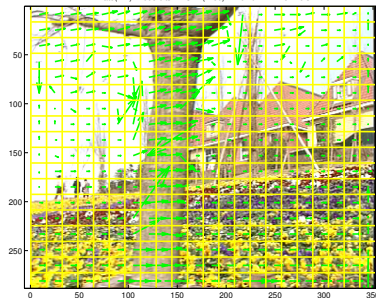
Parameters of motion estimation

Block size and shape

Rate(MV): 7938 bits – PSNR(Pred): 26.53 dB – Time: 26.6 s



Rate(MV): 1668 bits – PSNR(Pred): 24.51 dB – Time: 7.6 s



Block size 8×8

Block size 16×16



Parameters of motion estimation

Cost function

Several choices are possible for $d(\cdot, \cdot)$:

- ▶ SAD (Sum of Absolute Differences)

$$d(B_1, B_2) = \sum_{n,m} |B_1(n, m) - B_2(n, m)|$$

- ▶ SSD (Sum of Squared Differences)

$$d(B_1, B_2) = \sum_{n,m} [B_1(n, m) - B_2(n, m)]^2$$

SSD has the smallest MSE, but it is more complex and requires more bits (irregular vectors)



Parameters of motion estimation

Cost function

Regularized cost function:

- ▶ Instead of minimizing $d(v)$, one can minimize a regularized cost function J :

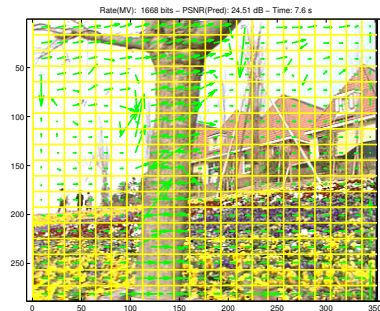
$$J(v) = d(v) + \lambda_{ME} r(v)$$

- ▶ $r(v)$ can be the coding cost (number of bits) of vector v
- ▶ Adding the regularization means that we can select a vector that does not achieve the best MSE, but costs less bit to be encoded
- ▶ The trade-off is driven by λ_{ME}
 - ▶ Small λ_{ME} means that the coding cost is not important and v must only minimize the distance d
 - ▶ Large λ_{ME} means that minimizing the coding cost is what matters, even at cost of a large “distortion” $d(v)$

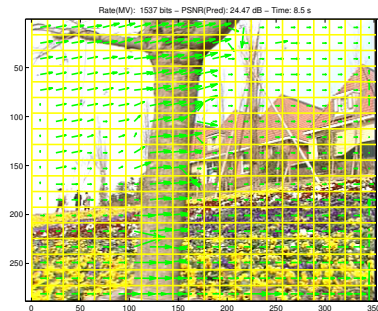


Parameters of motion estimation

Cost function



Non-regularized MVF

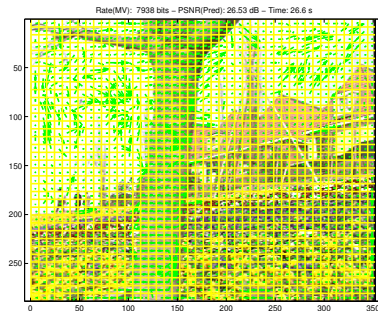


Regularized MVF

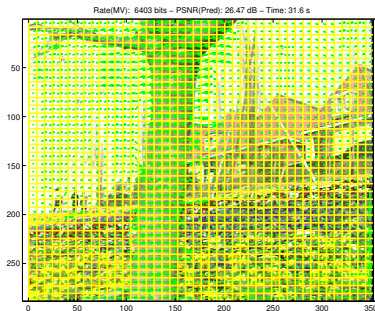


Parameters of motion estimation

Cost function



Non-regularized MVF



Regularized MVF



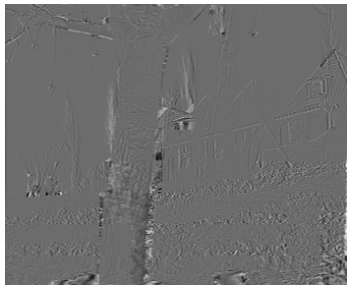
UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Cost function

Motion-compensated image



MC error



Regularized MVF, motion-compensated image

Regularized MVF, compensa-
tion error





Norm-based criteria: SSD

- ▶ If in Eq. (2) we choose $p = 2$ we obtain the criterion called *Sum of Squared Differences*, SSD:

$$J_{\text{SSD}}(i, j) = \sum_{(n, m) \in B_{p, q}} [f(n, m, k) - f(n - i, m - j, h)]^2$$

- ▶ For a given block size, the SSD minimizes the energy of the MC-ed prediction
- ▶ Therefor, the SSD is the best choice in terms of MVF quality



Norm-based criteria: SSD

However, the SSD has also several disadvantages:

- ▶ It is relatively complex to compute: the criterion d includes a multiplication
- ▶ The square operator highlights errors from outliers, resulting sometimes in irregular vectors, which in turns increase the entropy (coding cost) of the MVF
- ▶ It does not take into account global illumination changes
- ▶ The $p = 1$ norm reduces the impact of the first 2 problems





Norm-based criteria: SAD

- ▶ However, the SAD motion vector field is generally more regular than the one estimated with SSD
- ▶ The reason lies in **outliers**
- ▶ Outliers are pixels affected by large “noise”
- ▶ I.e., pixels not respecting the constant illumination hypothesis
- ▶ An outlier in position n, m contributes to the cost function with $|f(n, m, k) - f(n - i, m - j, h)|^p$
- ▶ Thus, if $p = 2$ a single outlier (or a few but concentrated in a small area) can affect the estimated motion vector



Norm-based criteria: examples

Reference image



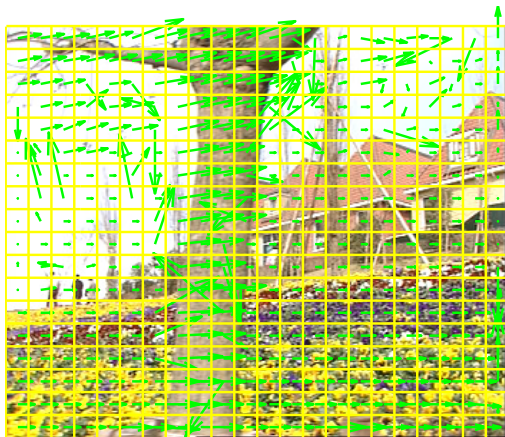
Current image





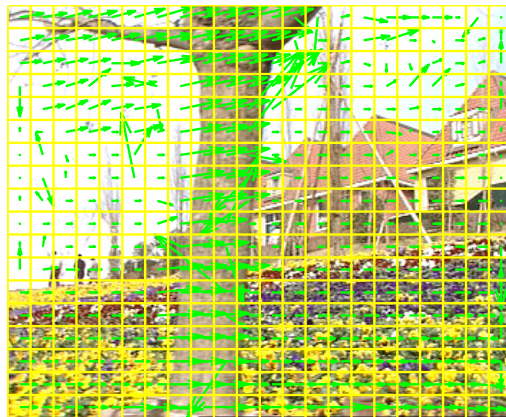
Norm-based criteria: examples

SSD Rate(MV): 2143 bits – PSNR(Pred): 22.46 dB



SSD

SAD Rate(MV): 2103 bits – PSNR(Pred): 22.30 dB



SAD



Norm-based criteria: examples

Motion-compesated image



SSD

Motion-compesated image

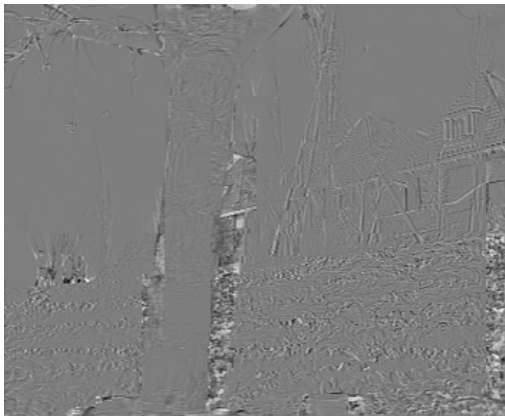


SAD



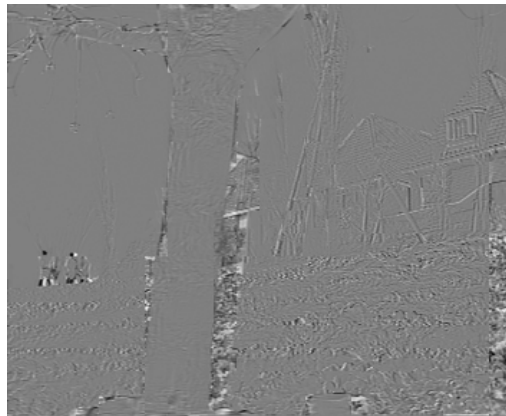
Norm-based criteria: examples

MC error



SSD

MC error

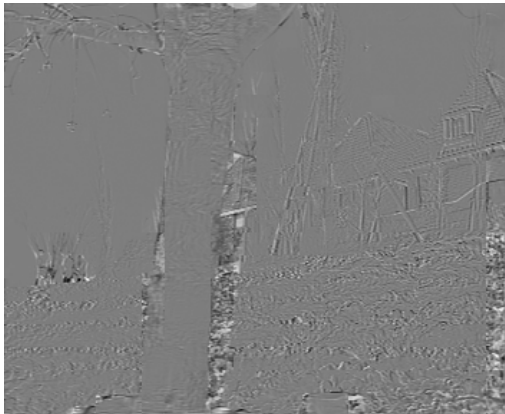


SAD

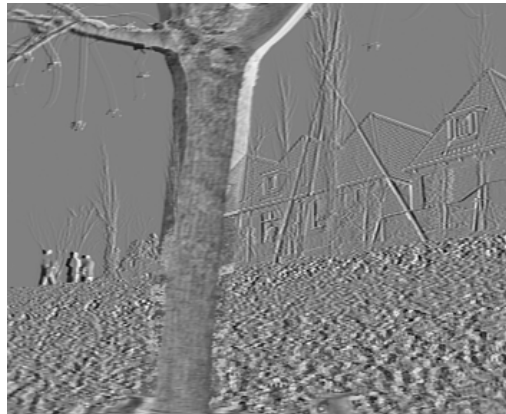


Norm-based criteria: examples

MC error



Difference image





Norm-based criteria

- ▶ These MVF are not always regular: the estimation in homogeneous regions depends finally on noise
- ▶ Advantage: minimization of the error energy
- ▶ Disadvantage: motion artifacts; high coding cost
- ▶ Solution: regularization
 - ▶ Large block sizes, $\mathcal{L}^1$ norm are a form of implicit regularization
 - ▶ We can resort to explicit regularization



Regularized norm-based criteria

- ▶ We modify the criterion (2) by penalizing vectors that are too much different from their neighbors
- ▶ This is accounted for in the function $R(i,j)$
- ▶ Thus we get:

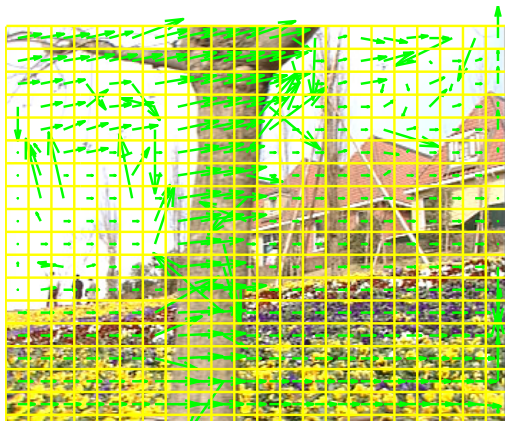
$$J_{\text{REG}}(i,j) = \|\mathbf{f}_k(B_{p,q}) - \mathbf{f}_h(B_{p-i,q-j})\|_p^p + \lambda R(i,j)$$

- ▶ For example, R could be the distance between (i,j) and a vector representing the neighborhood, such as the average or the median

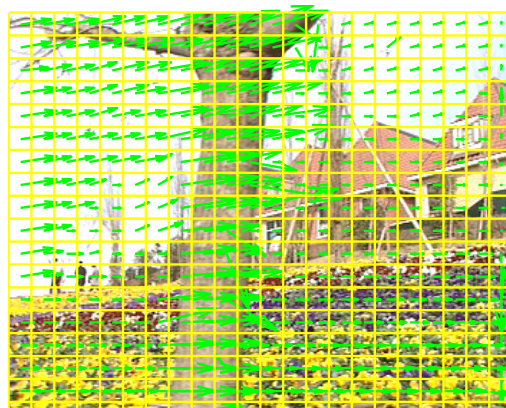


Regularized norm-based criteria: examples

SSD Rate(MV): 2143 bits – PSNR(Pred): 22.46 dB



SSD Rate(MV): 2008 bits – PSNR(Pred): 22.35 dB





Research strategies: Full search

- ▶ Naive solution: each vector (i, j) must be tested
 - ▶ We compute the criterion J as many times as possible motion vectors:
 $(N - P)(M - Q)$

- ▶ Often, it is enough to consider a smaller region centered in (p, q)

- ▶ We define a *search window* $\mathcal{W}$:

$$\mathcal{W} = \{-A, \dots, -1, 0, +1, \dots, A\} \times \{-B, \dots, -1, 0, +1, \dots, B\}$$

- ▶ Typically, $A = B$. If $n = 2A + 1$, there are n^2 candidate vectors in $\mathcal{W}$
- ▶ The *full search* motion estimation consists in computing J for each vector in $\mathcal{W}$, and taking the one that optimizes the criterion

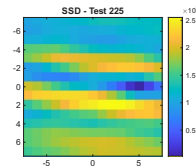
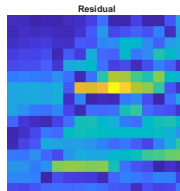
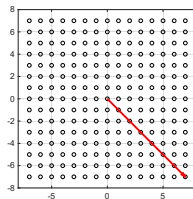
Full search: Visual Demo



Current Image (Target Block)



Reference (i=-7, j=7)

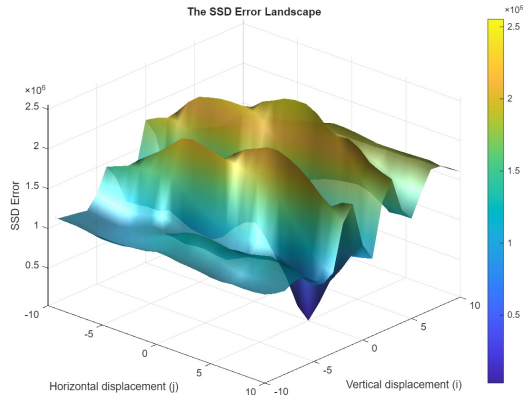


The cost function landscape



The search for the Motion Vector (i, j) is equivalent to finding the **global minimum** in a 3D cost-function surface.

- ▶ **Global Minimum:** The optimal vector (lowest residual energy).
- ▶ **Local Minima:** Risks for fast search algorithms (traps).
- ▶ **Flat areas:** Low texture, motion estimation becomes ambiguous.





Research strategies: Fast methods

- ▶ Full search demands n^2 computations in order to find the best vector limited to a displacement of $\pm A$ pixels (horizontally and vertically)
- ▶ Fast methods allow to find motions with the same amplitude $\pm A$ pixels with less than n^2 computations
- ▶ The basic idea is to test a subset of vectors
- ▶ On the other hand, fast methods no longer assure the optimal MV



Research strategies: Three steps search (3SS)

- ▶ The 3SS is based on the assumption that the error function is **unimodal**
- ▶ This implies a single global minimum and no local minima that could "trap" the search
- ▶ **Risk:** If the surface is complex (multimodal), fast searches may converge to a local minimum, missing the best match
- ▶ Successive steps with decreasing stride $(D, D/2, \dots, 1)$
- ▶ Tests 4 or 8 points around the center at each step

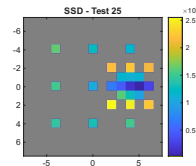
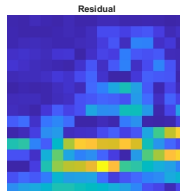
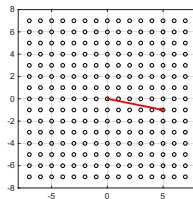


Three steps search: Visual Demo

Current Image (Target Block)



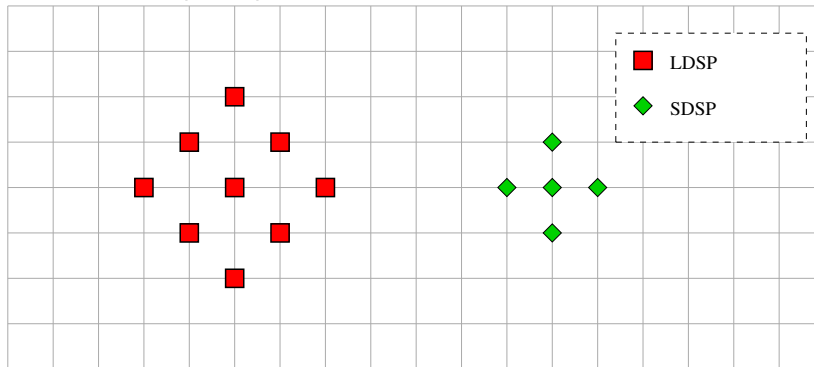
Reference (i=-1, j=5)





Research strategies: Diamond search

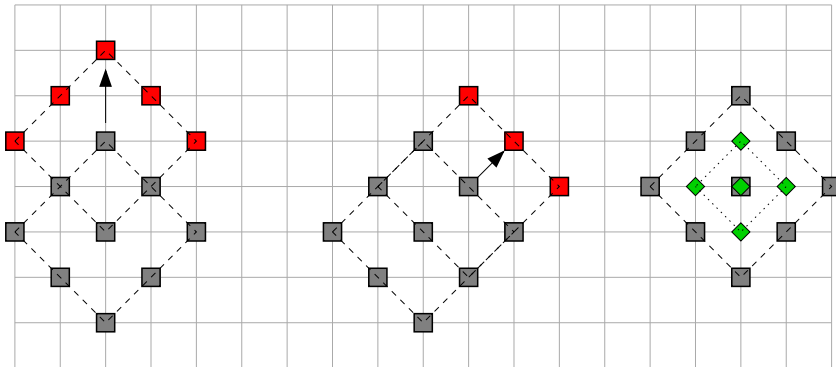
- ▶ Uses a diamond-shaped pattern (Large Diamond Search Pattern - LDSP).
- ▶ The pattern "moves" until the minimum is at the center
- ▶ Final refinement with a Small Diamond (SDSP)
- ▶ 3 to 5 new points per iteration (9 for the first)





Research strategies: Diamond search

- ▶ Uses a diamond-shaped pattern (Large Diamond Search Pattern - LDSP).
- ▶ The pattern "moves" until the minimum is at the center
- ▶ Final refinement with a Small Diamond (SDSP)
- ▶ 3 to 5 new points per iteration (9 for the first)



Diamond search: Visual Demo

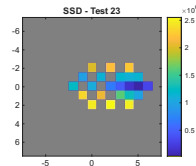
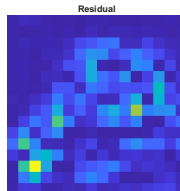
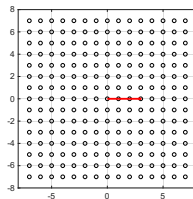


UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Current Image (Target Block)



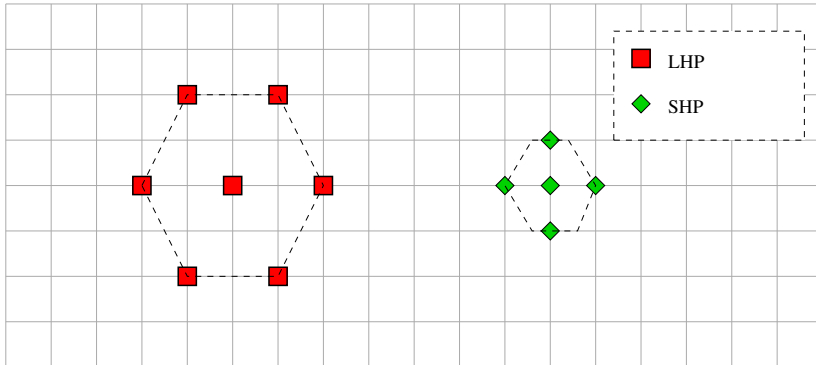
Reference (i=0, j=3)





Research strategies: Hexagon search

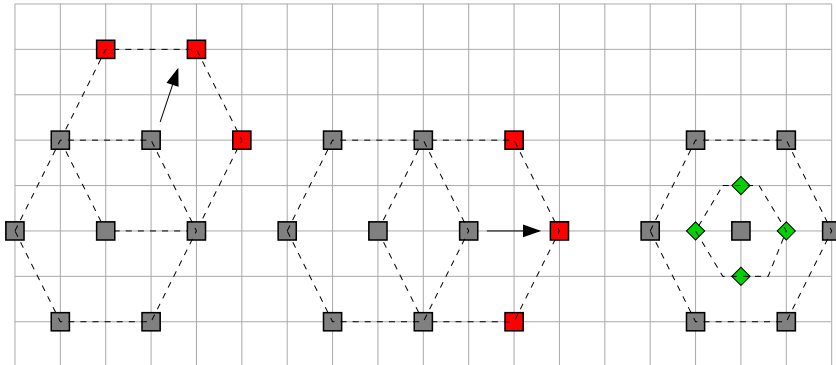
- ▶ Evolution of Diamond Search (H.264/AVC)
- ▶ More regular shape (6-sided) for more uniform displacement speed
- ▶ Fewer computations per step: only 3 new points if the center is not the minimum





Research strategies: Hexagon search

- ▶ Evolution of Diamond Search (H.264/AVC)
- ▶ More regular shape (6-sided) for more uniform displacement speed
- ▶ Fewer computations per step: only 3 new points if the center is not the minimum



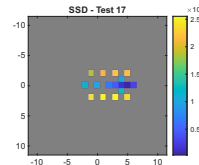
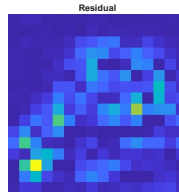
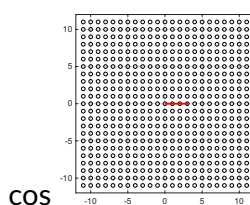


Hexagon search: Visual Demo

Current Image (Target Block)



Reference (i=0, j=3)





Comparison of Search Strategies

Performance on test case

In this example, all methods converge to the same optimal vector, but with vastly different number of tested vectors.

Strategy	Tests	$\Delta\%$
Full Search (FS)	225	Baseline
2D-Log Search	25	$\sim -89\%$
Diamond Search	23	$\sim -90\%$
Hexagon Search	17	$\sim -92\%$

Key Takeaways

- ▶ **Unbounded Search:** Diamond and Hexagon are **iterative**. Unlike FS, they aren't limited by a fixed n and can "follow" the motion across the frame.
- ▶ **Reliability:** While FS guarantees the global minimum, fast methods can get stuck in local minima (though rare in natural video).



Modern Fast Search: TZSearch

Beyond Fixed Patterns

The **Test Zone Search (TZSearch)** is the reference algorithm for recent standards. It adaptively switches between different strategies to handle large blocks (up to 64x64).

The Three Phases of TZSearch:

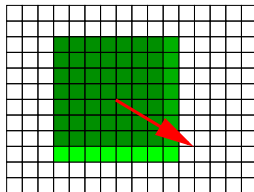
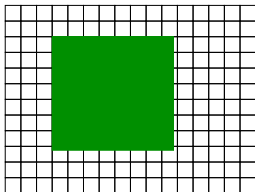
1. **Search Predictors:** Initial tests on vectors from spatial and temporal neighbors. If the error is low enough, the search stops early.
2. **Adaptive Loop:** If no predictor is good, it uses a Diamond/Square search with increasing steps.

Key Advantage

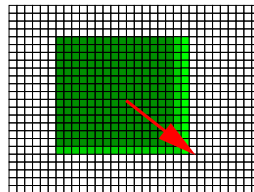
More robust against **local minima** compared to simple Diamond or Hexagon search.



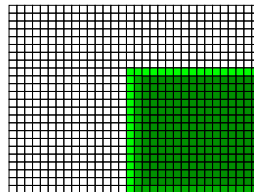
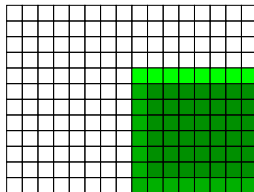
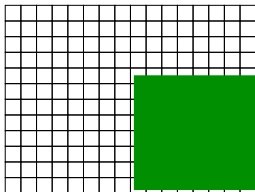
Block matching: subpixel precision



$v=(5,3)$



$v=(5.5,2.5)$

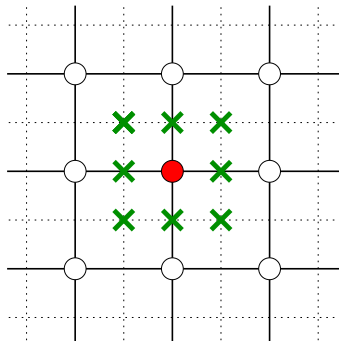




Block matching: subpixel precision

Hierarchical approach:

- ▶ First we find $(\hat{i}, \hat{j}) \in \mathbb{Z}^2$;
- ▶ then we test $(\hat{i} \pm \frac{1}{2}, \hat{j} \pm \frac{1}{2})$;
- ▶ then we verify the 1/4-pixel neighbors, etc.

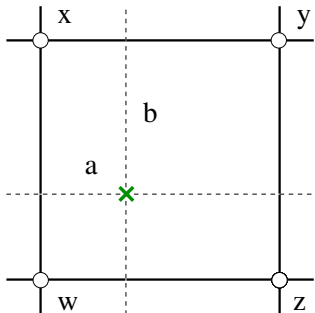


- Integer pixel positions
- Selected full-pixel vector
- × Tested half-pixel position



Block matching: subpixel precision

It is possible to evaluate $f(n+a, m+b, k)$ via interpolation:



$$f(n, m) = x \quad f(n+1, m) = y$$

$$f(n, m+1) = z \quad f(n+1, m+1) = w$$

Bilinear interpolation: horizontal and vertical average of pixels:

$$f(n+a, m+b) = (1-a)(1-b)x + a(1-b)y + (1-a)bz + abw$$

More complex interpolation techniques have been proposed (high order filters)



Block matching: variable block size

- ▶ Underlying hypothesis in BM: motion is homogeneous in a block
- ▶ It is not true if a block contains object's borders
- ▶ Solution 1: smaller blocks
 - ▶ Improved precision
 - ▶ Increased complexity
 - ▶ Increased coding cost (number of motion vectors per image)
- ▶ Solution 2: variable-size blocks
 - ▶ Additional coding cost and complexity: only if necessary
 - ▶ Hierarchical approach: a block is split if the value of the criterion is too big
 - ▶ This solution is adopted in recent video coding standards



Block matching: variable block size

Input: Frame F , Ref. frame R , Initial block size
 B, λ

Output: Motion vectors and partitioning
for each block b of size B in F do

 Calculate $J(v) = D + \lambda R$ for b ;

 Divide b into four sub-blocks;

for each sub-block s_i do

 | Calculate $J_i(v) = D_i + \lambda R_i$ for s_i ;

end

$J_{sub} = \sum_{i=1}^4 J_i(v)$;

if $J_{sub} < J(v)$ then

 | Apply algorithm to each sub-block;

else

 | Keep b & store MV;

end

end

Where:

- ▶ D : Distortion measure (e.g., SAD, SSD)
- ▶ R : Rate measure (bits to encode motion vector)
- ▶ λ : Lagrange multiplier for rate-distortion optimization
- ▶ v : Motion vector

Note:

- ▶ The algorithm is applied recursively to sub-blocks
- ▶ Termination condition: minimum block size or $J(v)$ no longer decreases
- ▶ The process balances between motion estimation accuracy and coding efficiency

Outline



Introduction

Variational methods

Block-matching methods

Criteria & Strategy

Advanced Techniques

Parametric methods

Deep Learning methods

Conclusions



Definitions

- ▶ Using parametric models means that the motion vector field is modeled as a close-form function of the pixel position
- ▶ The degrees of freedom are the *parameters* of the mathematical function
- ▶ The parametric model can be unique for all the scene (global motion estimation); or one can use a different parametric model per region
- ▶ In turns, the region can be an object (in this case, we first need to *segment* the image to find the objects) or a pre-determined set of pixels, e.g., a block



Definitions

- ▶ Block-matching methods can be seen as a special form of parametric methods, where the MVF is a block-wise constant function, with two free parameters per block (pure translation)
- ▶ The regions are the blocks
- ▶ More general models can be conceived
 - ▶ Less parameters → Robust estimation, but less generality, can miss the complexity of motion
 - ▶ More parameters → Prone to overfitting, more complex but also more general



Translational model

- ▶ The simplest motion model is the translational one
- ▶ This model corresponds to the case of a rigid object that translates orthogonally to the optic axis without rotations
- ▶ Let $\mathbf{v}(p)$ be the instantaneous velocity in position $p = [x, y]^T$ on the image plane
- ▶ Components $\mathbf{v}_x$ and $\mathbf{v}_y$ of $\mathbf{v}$ do not vary with p , they depend on camera parameters and on object motion
- ▶ In other words, the model is a constant vector (two parameters). We can write:

$$\mathbf{v}(p) = \begin{bmatrix} \mathbf{v}_x \\ \mathbf{v}_y \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

- ▶ This model is only characterized by two parameters, b_1 et b_2



Affine model

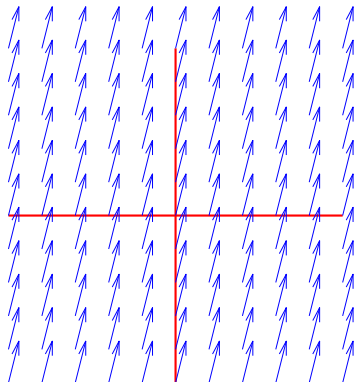
- ▶ The affine model is more complex, but it can represent a large variety of motions
- ▶ Its equation is:

$$\mathbf{v}(p) = \mathbf{b} + Bp = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} + \begin{bmatrix} b_3 & b_4 \\ b_5 & b_6 \end{bmatrix} p$$

- ▶ It includes translation as a special case when $B = 0$
- ▶ It is based on 6 parameters, two for $\mathbf{b}$ and four for B
- ▶ The relatively small number of parameters makes their estimation robust



Affine models: examples



Translation:

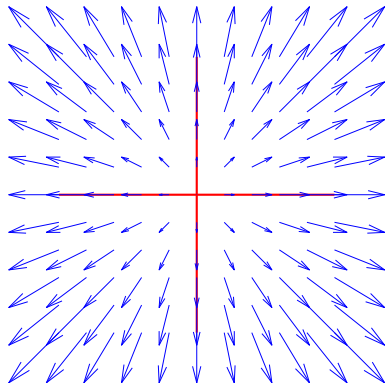
$$\mathbf{b} = \begin{bmatrix} 0.5 \\ 2 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\mathbf{v}_x = 0.5$$

$$\mathbf{v}_y = 2$$



Affine models: examples



Zoom in:

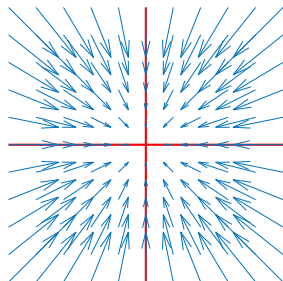
$$\mathbf{b} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

$$\mathbf{v}_x = 0.5x$$

$$\mathbf{v}_y = 0.5y$$



Affine models: examples



Zoom out:

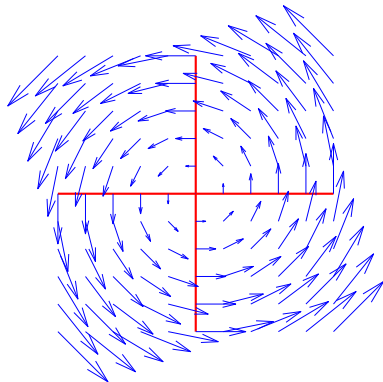
$$\mathbf{b} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} -0.5 & 0 \\ 0 & -0.5 \end{bmatrix}$$

$$\mathbf{v}_x = -0.5x$$

$$\mathbf{v}_y = -0.5y$$



Affine models: examples



Rotation:

$$\mathbf{b} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 0 & -0.5 \\ 0.5 & 0 \end{bmatrix}$$

$$\mathbf{v}_x = -0.5y$$

$$\mathbf{v}_y = 0.5x$$



Affine models: examples

A Compact Representation: Despite the visual complexity, the entire motion field is governed by only **6 parameters**, which replace the hundreds of individual vectors that would be needed with block-matching

What this example shows: The transformation in the figure is a combination of:

1. **Rotation:** The camera tilt.
2. **Zoom In:** The magnifying effect.
3. **Translation:** Global shift.



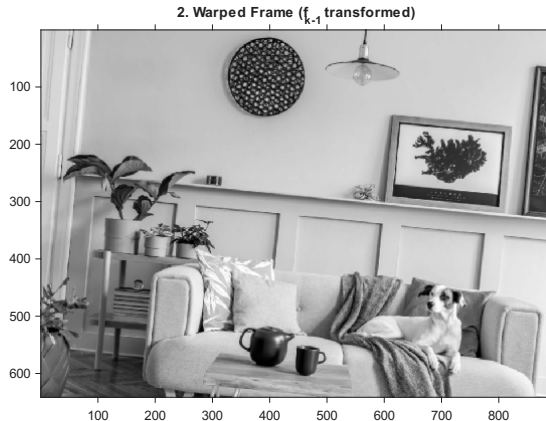


Affine models: examples

A Compact Representation: Despite the visual complexity, the entire motion field is governed by only **6 parameters**, which replace the hundreds of individual vectors that would be needed with block-matching

What this example shows: The transformation in the figure is a combination of:

1. **Rotation:** The camera tilt.
2. **Zoom In:** The magnifying effect.
3. **Translation:** Global shift.



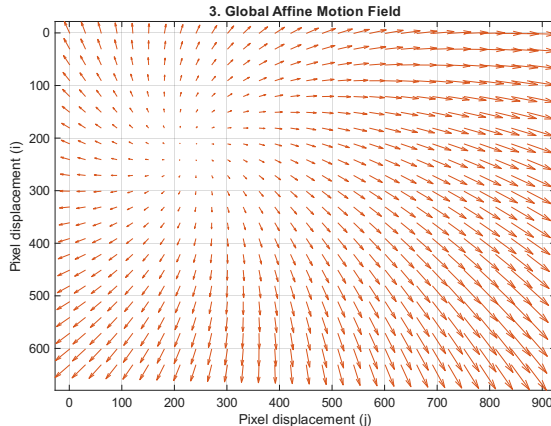


Affine models: examples

A Compact Representation: Despite the visual complexity, the entire motion field is governed by only **6 parameters**, which replace the hundreds of individual vectors that would be needed with block-matching

What this example shows: The transformation in the figure is a combination of:

1. **Rotation:** The camera tilt.
2. **Zoom In:** The magnifying effect.
3. **Translation:** Global shift.





Indirect estimation

- ▶ How to estimate the parameters of the model?
- ▶ A first approach consists in first estimating a dense field (e.g. with optical flow), and then deduce the global motion from it solving an optimization problem, typically with least square techniques
- ▶ Let $u(n, m)$ et $v(n, m)$ be the estimated dense field components
- ▶ Let $u_\pi(n, m)$ et $v_\pi(n, m)$ be some dense fields depending on the parameter set π
 - ▶ For example, for affine models, $\pi = [b_1 \ b_2 \ b_3 \ b_4 \ b_5 \ b_6]$
- ▶ Indirect estimation amount to:

$$\pi^* = \arg \min_{\pi} \sum_{n, m \in \mathcal{R}} [u(n, m) - u_\pi(n, m)]^2 + [v(n, m) - v_\pi(n, m)]^2$$



Indirect estimation

- ▶ Indirect estimation looks for the parameters that make the model the closest to the estimated MVF
- ▶ This distance minimization may be solved with a gradient descent
- ▶ Disadvantage:
 - ▶ Strong dependency from the first dense estimation
 - ▶ It needs a support region $\mathcal{R}$ with homogeneous motion



Direct estimation

- ▶ With Direct Estimation, the parameters are introduced in the estimation phase
- ▶ For example, OF equation becomes:

$$\pi^* = \arg \min_{\pi} \sum_{n,m \in \mathcal{R}} [u_{\pi}(n, m)f_x(n, m) + v_{\pi}(n, m)f_y(n, m) + f_t(n, m)]^2$$

- ▶ Another approach consists in minimize the SSD or the SAD computed on the parametrized MVF:

$$\pi^* = \arg \min_{\pi} \sum_{n,m \in \mathcal{R}} [e(n, m)]^2 \quad \text{with}$$

$$e(n, m) = f(n - u_{\pi}(n, m), m - v_{\pi}(n, m), t - 1) - f(n, m, t)$$

- ▶ SAD-based block matching can be seen as a special case of direct parametric estimation where the motion model is a pure translation, *i.e.* special case of affine model with $B = 0$

Outline

Introduction

Variational methods

Block-matching methods

Criteria & Strategy

Advanced Techniques

Parametric methods

Deep Learning methods

Conclusions



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Deep Learning for Motion Estimation

The Shift in Paradigm

Traditional methods (Differential or Block-based) rely on hand-crafted models. Deep Learning approaches treat motion estimation as a supervised or unsupervised learning problem, typically implemented via Convolutional Neural Networks (CNNs).

Key Requirements:

- ▶ Specialized architectures for pixel-level correspondence.
- ▶ Large-scale datasets for training (Synthetic or Real).
- ▶ High computational power for inference (GPUs).



Training Challenges: Ground Truth Availability

Optical flow ground truth is hard to obtain for real-world sequences.

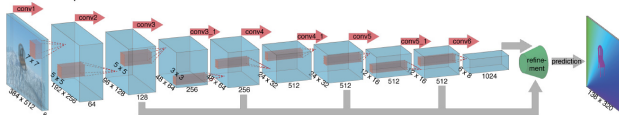
Data Solutions

- ▶ **Synthetic Datasets:** Using computer graphics (e.g., FlyingChairs, Sintel) to provide perfect labels.
- ▶ **Data Augmentation:** Geometric and photometric transformations to increase diversity.
- ▶ **Unsupervised Learning:** Minimizing the photometric error between the current frame and the warped reference.

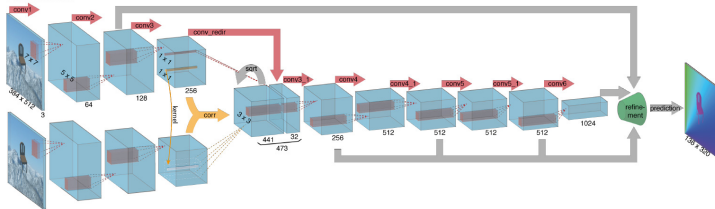


First Generation: FlowNet (2015)

FlowNetSimple



FlowNetCorr



Key Concepts:

- ▶ First end-to-end CNN for motion.
- ▶ **FlowNetS:** Simple concatenation.
- ▶ **FlowNetC:** Explicit correlation layer.

Dosovitskiy et al. (2015). FlowNet:
Learning Optical Flow with
Convolutional Networks



FlowNet: Technical Insights & Limits

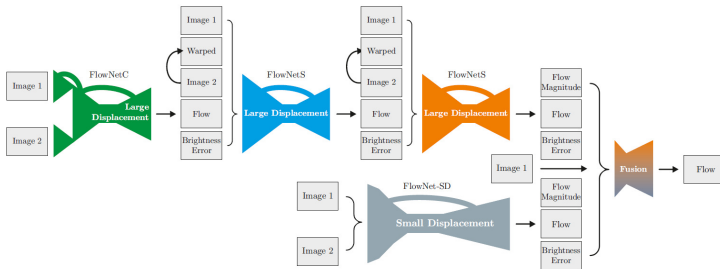
Performance & Bottlenecks

While revolutionary, the first FlowNet models struggled with small displacements and fine details compared to classical variational methods.

- ▶ **Data dependency:** Performance is heavily tied to the *Flying Chairs* dataset quality.
- ▶ **Inference Speed:** ≈ 10 -100 FPS on a mid-range GPU (much faster than traditional global optimization).
- ▶ **Weakness:** Large errors in regions with repetitive patterns due to the limited receptive field of early CNN layers.



Second Generation: FlowNet 2.0



Improvements:

- ▶ **Stacked architectures.**
- ▶ **Specific sub-networks for small motions.**
- ▶ **Advanced training on synthetic data.**

Ilg et al. (2017). FlowNet 2.0:
Evolution of Optical Flow Estimation
with Deep Networks



FlowNet 2.0: Complexity vs. Accuracy

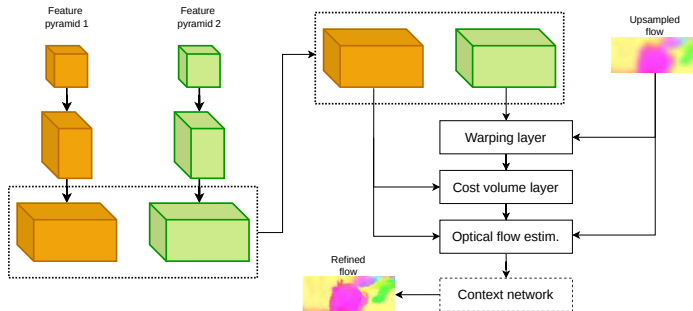
Architectural Refinement

FlowNet 2.0 stacks multiple FlowNet modules, significantly reducing the End-Point Error (EPE) at the cost of increased complexity.

- ▶ **Model Size:** Large memory footprint due to the "stacking" of multiple networks (FlowNetC + FlowNetS).
- ▶ **Schedule:** Introduced a complex training schedule (multi-stage training on different datasets).
- ▶ **Small Displacement Module:** A specialized sub-network for slow motions, where original FlowNet failed.
- ▶ **Efficiency:** Trade-off between FlowNet2 (most accurate) and FlowNet2-s (faster, lighter).



Third Generation: PWC-Net (2018)



Hybrid Design:

- **Pyramidal** feature extraction.
- Feature **Warping**.
- Local **Cost Volume** calculation.

Sun et al. (2018). PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume



PWC-Net: Efficiency & Hardware Friendly

The Power of Priors

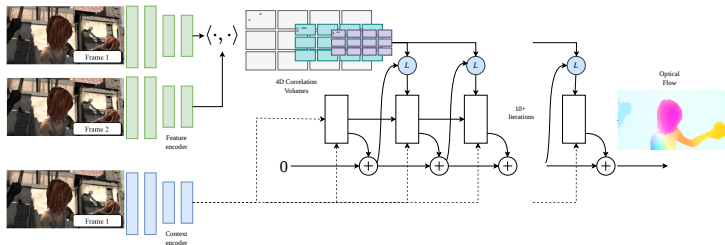
By using Cost Volumes and Warping, PWC-Net achieves SOTA accuracy with 17x fewer parameters than FlowNet2.

- ▶ **Complexity:** Extremely efficient. The Cost Volume is computed locally (limited search range), keeping the GPU memory usage low.
- ▶ **Execution Time:** $\approx 30\text{ms}$ for Sintel resolution images.
- ▶ **Evaluation:** Outstanding results on *KITTI* benchmarks, proving high generalization from synthetic to real-world driving sequences.

SOTA: RAFT (2020)



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Innovation:

- ▶ **Recurrent update (GRU).**
- ▶ **All-pairs correlation volume.**
- ▶ **No spatial pyramids.**

Teed & Deng (2020). RAFT: Recurrent All-Pairs Field Transforms for Optical Flow



RAFT: Iterative Convergence & Precision

The New Standard

RAFT maintains a high-resolution flow field throughout the process, avoiding the loss of detail typical of coarse-to-fine pyramids.

- ▶ **Iterative Logic:** The number of GRU iterations can be tuned at inference time (Trade-off: Speed vs. Quality).
- ▶ **Complexity:** Computing all-pairs correlation is memory-intensive, but optimized via a "Correlation Pyramid".
- ▶ **Generalization:** Unprecedented "zero-shot" generalization (trained on synthetic data, works perfectly on real video).
- ▶ **Inference:** Slower than PWC-Net but significantly more precise on thin structures and motion boundaries.



Synthesis of Deep Learning Architectures

Evolutionary Milestones

Deep Learning for Motion Estimation has evolved through different paradigms, progressively integrating physical priors and iterative refinement logic.

- ▶ **FlowNet (2015/17):** Demonstrated the feasibility of direct dense flow regression using end-to-end CNNs
- ▶ **PWC-Net (2018):** Improved efficiency and accuracy by embedding classic vision concepts: Pyramids, Warping, and Cost Volumes
- ▶ **RAFT (2020):** Reached state-of-the-art precision through iterative updates using a Gated Recurrent Unit (GRU) and all-pairs correlation

DL-based ME: Perspectives and Challenges

Analysis vs. Compression

While DL methods dominate analysis tasks (Optical Flow), their integration into real-time systems and video coding standards requires balancing accuracy with practical constraints

Main Advantages:

- ▶ Robustness to occlusions and large displacements
- ▶ High precision in complex textureless areas

Current Challenges:

- ▶ **Generalization:** Performance drops on data differing from training sets
- ▶ **Computational Cost:** High memory and power requirements for SOTA

Outline

Introduction

Variational methods

Block-matching methods

Criteria & Strategy

Advanced Techniques

Parametric methods

Deep Learning methods

Conclusions



UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione



Motion Estimation: Wrap-up

- ▶ Motion Estimation allows to extract motion information from videos
- ▶ It focuses on *optical flow* rather than physical motion (other sensors can be used to assess the physical motion)
- ▶ Block-matching: conceptually simple, commonly used for video compression
- ▶ Neural networks: emerging framework, very effective for analysis tasks

